Kusanagi Kajiki

ICS/SCADA Passive Network Discovery & Security Analysis Tool

A modern rewrite of NSA's GRASSMARLIN

Version 1.5 | March 2026

David — The Security Lead

431	Tests Passing
95	Tauri Commands
11	Rust Crates
10	Deep-Parsed Protocols
40+	ATT&CK Detection Rules
25k+	Lines of Rust

1. Introduction

Kusanagi Kajiki is a ground-up rewrite of the NSA's archived GRASSMARLIN tool, rebuilt from scratch with a Rust backend (Tauri 2.0) and SvelteKit frontend. It passively discovers and maps Industrial Control System (ICS) and SCADA network devices by analyzing network traffic captures and integrating data from external security tools.

In operational technology environments, **active scanning can crash PLCs and disrupt physical processes**. Assessors need visibility into OT networks without generating a single packet. Kusanagi Kajiki operates in passive-only mode — it observes captured traffic, identifies devices and protocols, maps network topology, and flags security concerns, all without touching the production network.

Target audience: OT security assessors, ICS incident responders, compliance auditors, and anyone who needs to understand what's on an industrial network without scanning it.

1.1 What Makes It Different

Kusanagi Kajiki achieves full feature parity with GRASSMARLIN 3.2 and extends well beyond it. Key additions include: MITRE ATT&CK; for ICS detection (40+ rules across 10+ technique IDs), ICS malware behavioral signatures (FrostyGoop, PIPEDREAM, Industroyer2), Purdue Model enforcement, IEC 62443/NIST 800-82/NERC CIP compliance mapping, deep protocol analysis for 10 ICS protocols, Zeek/Suricata/Wazuh log ingestion, microsegmentation recommendation with enforcement config generation, and professional PDF reporting.

1.2 By the Numbers

Metric	Count	Metric	Count
Rust tests	431	Analysis modules	16
Tauri commands	95	External tool importers	7
YAML signatures	31	ATT&CK; detection rules	40+
Deep-parsed protocols	10	Default credential entries	35
Rust crates	11	Frontend components	20
Detected protocols	19+	Lines of Rust	25,000+

2. Installation

2.1 Prerequisites

Rust >= 1.77, Node.js >= 22, and platform-specific libpcap development headers.

2.2 Platform Dependencies

Platform	System Dependencies	Notes
Fedora/RHEL	libpcap-devel webkit2gtk4.1-devel libsoup3-devel javascriptcoregtk4.1-devel	Primary dev platform
Ubuntu/Debian	libpcap-dev libwebkit2gtk-4.1-dev libappindicator3-dev librsvg2-dev patchelf	
macOS	brew install libpcap + Xcode CLI tools	
Windows	VS C++ Build Tools + Npcap (WinPcap mode) + Npcap SDK	Set LIB env var to SDK Lib/x64 path

2.3 Build Steps

```
git clone https://github.com/TheSecurityLead/KusanagiNoKajiki.git
cd KusanagiNoKajiki
npm install
npm run build
npm run tauri dev        # development mode
npm run tauri build      # production binary
```

2.4 Verify Installation

```
cd src-tauri && cargo test --all          # 431 tests
cargo clippy --all -- -D warnings         # zero warnings
cd .. && npm run check                    # frontend types
```

2.5 Windows-Specific Notes

Install Visual Studio C++ Build Tools with the 'Desktop development with C++' workload. Install Npcap with 'WinPcap API-compatible Mode' checked. Download the Npcap SDK and set the LIB environment variable to the SDK's Lib\x64 directory. Do NOT run npm audit fix --force.

3. Quick Start Guide

5-minute workflow: Import a PCAP → View the topology → Inspect devices → Run security analysis → Run segmentation analysis → Export enforcement configs.

Step 1: Launch the application

Run the built binary or use npm run tauri dev.

Step 2: Import a PCAP file

Open the Capture tab and click Import PCAP File(s). Multi-file selection is supported.

Step 3: Explore the Connection Tree

The left panel shows an expandable tree: nodes, connections, and individual packets.

Step 4: View the Logical Topology

Switch to the Topology tab. The graph uses fcose layout with subnet grouping. Switch to Purdue layout for level-banded view.

Step 5: Inspect a device

Click any node or open the Inventory tab. See vendor ID, protocols, confidence, deep parse details, CVE matches, SNMP identity, and default credential warnings.

Step 6: Run Security Analysis

Open the Analysis tab and click Run Analysis. The engine detects ATT&CK; techniques, assigns Purdue levels, flags anomalies, checks for ICS malware signatures, and maps compliance requirements.

Step 7: Run Segmentation Analysis

Open the Segmentation tab and click Run Segmentation Analysis. This generates identity-based policy groups, IEC 62443 zone/conduit recommendations, a least-privilege communication matrix, enforcement configs in 5 formats, and a policy simulation.

Step 8: Export results

Use the Export tab for PDF reports, CSV/JSON, SBOM, STIX 2.1. Use the Segmentation > Enforcement tab to export Cisco ACLs, Suricata rules, or JSON policy directly.

4. Core Features

4.1 Protocol Detection (10 Deep-Parsed)

Protocol	Port(s)	Detection	Device ID
Modbus TCP	502	Deep parse	FC 43/14 vendor/product
DNP3	20000	Deep parse	Address, master/outstation
EtherNet/IP + CIP	44818, 2222	Deep parse	ListIdentity: vendor/product/serial
S7comm	102	Deep parse	TSAP rack/slot, SZL identity
BACnet/IP	47808	Deep parse	I-Am: device instance/vendor
IEC 60870-5-104	2404	Deep parse	Station address, type IDs
PROFINET DCP	34962-34964	Deep parse	Name/vendor/device ID/role

Protocol	Port(s)	Detection	Device ID
LLDP	—	Deep parse	Chassis/port/system/mgmt addr
SNMP	161, 162	Deep parse	sysDescr/sysName/community
Redundancy	—	Deep parse	MRP/RSTP/HSR/PRP/DLR

An additional 9+ protocols (OPC UA, MQTT, HART-IP, Foundation Fieldbus, GE SRTP, Wonderware SuiteLink, HTTP/S, DNS, SSH, RDP) are detected by port and YAML signature matching.

4.2 Topology Views

- **Logical View** — fcose or Purdue layout, compound subnet nodes, drift highlighting, PNG export
- **Physical View** — Cisco/Juniper/HP-Aruba/generic + traffic-inferred topology, ring redundancy
- **Mesh View** — All-to-all connection matrix with protocol/time filters
- **Timeline Scrubber** — Chronological topology replay with playback controls

4.3 Device Identification

- 31 YAML signatures covering 13+ OT protocols and vendor-specific patterns
- IEEE OUI database (~30k MAC-to-vendor entries)
- CIP, PROFINET, and BACnet vendor ID lookup tables
- GeoIP country identification for public IPs
- SNMP device identity extraction (sysDescr, sysName, sysLocation)
- LLDP infrastructure identification (chassis, port, system capabilities)
- 5-level confidence scoring: port (1) < pattern (2) < OUI (3) < payload (4) < deep parse (5)

4.4 External Tool Integration (7 Sources)

Source	Format	Notes
Zeek	TSV (conn/modbus/dnp3/s7comm)	Per-device drill-down
Suricata	EVE JSON	Flow + alert events
Nmap	XML	Host/port/service discovery
Masscan	JSON	Fast port scanning results
Wazuh	JSON	HIDS/SIEM alert correlation
Siemens SINEMA	CSV	Server inventory import
Siemens TIA Portal	XML	Project topology import

5. Security Analysis

The Analysis tab runs 16 security analysis modules with a single click. Results populate across multiple sub-tabs: Summary, Findings (ATT&CK;), Purdue Model, Anomalies, Baseline Drift, Switch Security, and Compliance.

5.1 MITRE ATT&CK; for ICS

40+ automated detection rules across 10+ ATT&CK; for ICS technique IDs. Detections include base rules (Phase 10), protocol-specific rules (Phase 12), network-level detections (flat network, cleartext OT, internet exposure), and 18 context-aware detections using full network state.

Technique	ID	Protocols	Severity
Unauthorized Command	T0855	Modbus, EtherNet/IP, S7comm, BACnet, IEC 104	Critical/High
Modify Program	T0836	EtherNet/IP (CIP File class)	Critical
Program Download	T0843	S7comm	Critical
Program Upload	T0845	S7comm	High
Data Destruction	T0809	S7comm (PI Service)	Critical
Device Restart/Shutdown	T0816	S7comm, BACnet, IEC 104	Critical

Technique	ID	Protocols	Severity
Denial of Service	T0814	Modbus, IEC 104	High/Medium
Alarm Suppression	T0856	DNP3, BACnet	High/Medium
Communication Control	T0811	BACnet	High
Remote Discovery	T0846	Any, EtherNet/IP, PROFINET	High/Medium
Remote Services	T0886	Cross-Purdue	Medium

5.2 ICS Malware Detection

Behavioral detection signatures for three major ICS malware families: **FrostyGoop** (Modbus write patterns targeting building automation), **PIPEDREAM/INCONTROLLER** (multi-protocol attack behavioral signatures), and **Industroyer2** (IEC 104 command injection patterns).

5.3 Purdue Model & Compliance

Automatic Purdue level assignment (L0-L4) based on protocol roles and behavior. Cross-zone violations detected and reported. Compliance mapping covers IEC 62443 (zones, conduits, security levels), NIST SP 800-82, and NERC CIP controls.

5.4 Additional Analysis Modules

- **Communication pattern analysis** — per-connection stats, jitter, periodicity, anomalies
- **CVE matching** — OT-focused database with vendor/product/firmware matching
- **Default credential warnings** — 35-entry ICS vendor credential database
- **Asset criticality scoring** — Critical/High/Medium/Low
- **Switch security assessment** — port security, BPDU guard, DHCP snooping analysis
- **Communication allowlisting** — flow classification + firewall rule generation

6. Microsegmentation Recommendation

New in Phase 15. Inspired by Elisity's identity-based microsegmentation platform, adapted for passive, on-prem OT assessments. Generates zone recommendations and enforcement configs from PCAP analysis alone — no cloud, no switch access required.

6.1 Overview

The Segmentation tab provides a complete microsegmentation recommendation workflow: identity-based policy group clustering, IEC 62443-aligned zone and conduit boundary recommendations, least-privilege communication rules, enforcement configuration export in five formats, and policy impact simulation. All derived from passively observed traffic.

6.2 Identity-Based Policy Groups

Assets are clustered into policy groups based on **what they are**, not where they sit on the network. The grouping algorithm uses five dimensions: Purdue level (primary), protocol role (master/slave, scanner/adapter), device category (PLC, HMI, historian), vendor family (conditional split by peer set disjointness), and communication community (for unclassified assets).

Each group receives an IEC 62443 Security Level target (SL1-SL4) and a criticality rating. Example groups: 'L1-PLC-Rockwell', 'L2-HMI-Stations', 'L3-Historians', 'L4-IT-Endpoints'.

6.3 Zone & Conduit Recommendations

Policy groups are organized into IEC 62443 zones aligned with the Purdue Model: Control Zone (L0/L1), Supervisory Zone (L2), Operations Zone (L3), DMZ Zone (L3.5), and Enterprise Zone (L4). Sub-zones are created when non-communicating policy groups exist within the same zone.

Conduits are generated for every observed inter-zone communication path with protocol rules, direction (uni/bidirectional), write/config operation flags, and risk notes. Cross-Purdue violations, missing DMZ boundaries, and flat network conditions are automatically flagged with severity-rated recommendations.

A **Zone Score** (0.0 to 1.0) quantifies segmentation quality based on the ratio of cross-Purdue violations to total inter-zone traffic.

6.4 Communication Matrix

For each zone pair, the engine generates the minimum set of allow rules covering observed legitimate traffic. Each rule includes protocol, port, direction, a risk classification (Low = read-only, Medium = writes, High = config operations), and a justification string citing the observation count, source/destination counts, and time range. Default action is deny-all.

6.5 Enforcement Config Export

Five output formats are generated simultaneously:

Format	Use Case
Cisco IOS Extended ACL	Most common OT switch platform
Cisco ASA ACL	Firewall-centric deployments (with object-groups)
Generic Firewall Table	Vendor-neutral TSV for manual implementation
Suricata Rules	IDS monitoring before hard enforcement
JSON Policy	Automation, SOAR, or custom integration

Each format includes per-rule justification comments, default deny, and application instructions. Configs can be previewed in the UI, exported to file, or copied to clipboard.

6.6 Policy Simulation

The simulation engine replays all observed traffic against the proposed policy. For each connection, it determines whether a matching zone-pair rule exists. Results include:

- **Block summary** — total allowed vs blocked connections, block percentage
- **Risk reduction score** — weighted by ATT&CK; finding severity (Critical=4x, High=3x)
- **Deployment score** — IEC 62443 SL-weighted policy coverage metric
- **Critical blocks** — blocked traffic to PLCs/safety systems requiring review
- **False positive candidates** — periodic read-only OT traffic flagged for rule addition

7. Reporting & Export

7.1 PDF Assessment Reports

Professional assessment reports generated via genpdf with executive summary, asset inventory, protocol analysis, findings table, and recommendations.

7.2 Export Formats

- **CSV/JSON** — assets, connections, topology data
- **SBOM** — CISA BOD 23-01 aligned software bill of materials
- **STIX 2.1** — threat intelligence bundles with observed indicators
- **Filtered PCAP** — export packets matching IP/port filters
- **Remediation priority list** — ranked CSV export
- **Communication allowlist** — classified flows + firewall rules
- **Enforcement configs** — Cisco ACL, Suricata, JSON (from Segmentation tab)

7.3 Session Management

Sessions are persisted to SQLite (WAL mode) and can be exported as portable .kkj ZIP archives. Baseline drift detection compares the current assessment against a saved session with a quantified drift score highlighting new, missing, and changed assets.

The Projects view enables engagement management with named projects, session scoping, and metadata tracking.

8. Architecture

8.1 System Overview

```
SvelteKit Frontend (Tauri Webview) — 20 components
  Topology · Inventory · Analysis · Segmentation · Export · Settings
═══════════════════════════════════════════════════════
Tauri IPC: 95 Commands + Event Streaming
═══════════════════════════════════════════════════════
Rust Backend (11 crates, 25k+ lines)
  gm-capture · gm-parsers · gm-signatures · gm-topology
  gm-db · gm-physical · gm-ingest · gm-analysis
  gm-report · gm-segmentation · commands
```

8.2 Data Pipeline

```
PCAP / Live Capture
  → gm-capture (L2-L4 parsing)
  → gm-parsers (protocol ID + deep parse for 10 protocols)
  → gm-signatures (31 YAML pattern matches)
  → gm-topology (petgraph network graph)
  → OUI / GeoIP enrichment
  → gm-analysis (ATT&CK; + Purdue + CVE + malware + compliance)
  → gm-segmentation (zones + matrix + enforcement + simulation)
  → gm-db (SQLite persistence)
  → gm-report (PDF / CSV / SBOM / STIX)
  → Frontend (Cytoscape topology, tables, charts)
```

8.3 Tech Stack

Rust Backend: Tauri 2.0, pcap, etherparse, petgraph, serde (JSON/YAML), tokio, thiserror, chrono, uuid, rusqlite (bundled), maxminddb, regex, quick-xml, genpdf, clap 4, zip.

Frontend: SvelteKit (Svelte 5), TypeScript (strict), Cytoscape.js + fcose layout, Tailwind CSS 4, CodeMirror 6, @tauri-apps/api + plugins (dialog, shell).

9. CLI Reference

```
kusanaginokajiki                        # Launch GUI
kusanaginokajiki --open capture.pcap    # Open PCAP on startup
kusanaginokajiki --open session.kkj     # Open session archive
kusanaginokajiki --import-pcap file.pcap # Import PCAP on startup
```

10. Troubleshooting

npm install fails with ERESOLVE: Clean install with pinned versions. Do NOT use npm audit fix --force.

linker 'link.exe' not found: Install Visual Studio Build Tools with 'Desktop development with C++' workload. Restart terminal.

Npcap linking errors on Windows: Ensure WinPcap API-compatible mode is checked. Set LIB to Npcap SDK Lib/x64 directory.

Permission denied on live capture (Linux): Run: sudo setcap cap_net_raw,cap_net_admin=eip target/release/kusanaginokajiki

Zone.Identifier files in git: Windows NTFS artifacts. Blocked by .gitignore. Remove: find . -name '*Zone.Identifier' -delete

Large PCAP causes crash on Windows: Known issue with >200MB files on Communication Patterns tab. Use filtered PCAP export to reduce size.

Appendix A: Supported Protocols

Protocol	Port(s)	Depth	Standard/Vendor
Modbus TCP	502	Deep parse	Multi-vendor (Schneider, etc.)
DNP3	20000	Deep parse	IEEE 1815 (utilities)
EtherNet/IP + CIP	44818, 2222	Deep parse	ODVA (Rockwell)
S7comm	102	Deep parse	Siemens S7 PLCs
BACnet/IP	47808	Deep parse	ASHRAE (building auto.)
IEC 60870-5-104	2404	Deep parse	Power grid SCADA
PROFINET DCP	34962-34964	Deep parse	Siemens/PROFIBUS Int'l
LLDP	—	Deep parse	IEEE 802.1AB
SNMP	161, 162	Deep parse	Multi-vendor
Redundancy	—	Deep parse	MRP/RSTP/HSR/PRP/DLR
OPC UA	4840	Port + Sig	OPC Foundation
MQTT	1883, 8883	Port	IIoT gateways
HART-IP	5094	Port	Process instrumentation
FF HSE	1089-1091	Port	Process automation
GE SRTP	18245-18246	Port + Sig	GE Automation
Wonderware SuiteLink	5007	Port + Sig	AVEVA

Appendix B: Beyond GRASSMARLIN

Capability	GRASSMARLIN 3.2	Kusanagi Kajiki
Security analysis	None	ATT&CK; (40+ rules) + Purdue + malware
Microsegmentation	None	Zone/conduit rec. + 5 enforcement formats
Deep protocol parsing	Limited	10 protocols with full dissection
External tool integration	None	7 sources (Zeek, Suricata, Wazuh, etc.)
Compliance mapping	None	IEC 62443, NIST 800-82, NERC CIP
ICS malware detection	None	FrostyGoop, PIPEDREAM, Industroyer2
CVE matching	None	OT-focused database
Reporting	None	PDF + CSV + SBOM + STIX 2.1
Architecture	Monolithic Java	11 Rust crates + SvelteKit